



Carnegie Mellon University

Master of
Software Engineering

LG Architecture Training Program

Software Architecture Project
Introduction Timegrapher Project

What This Lecture Is (and Is Not)

This is NOT:

- A detailed requirements walkthrough
- A coding tutorial
- A signal-processing lecture
- A UI design review

This IS:

- A discussion about architectural thinking
- A guide to approaching ambiguity
- A guide to managing technical risk
- A guide to making good architectural decisions under pressure

Why This Project?

- Learn by doing
 - This project gives you a chance to practice "architectural thinking"
 - This is not a "coding" project ... although you will certainly need to analyze and write code
 - It's about analyzing architectural tradeoffs and reducing uncertainty early in the project
- Technical Uncertainty
 - There will be lots of technical uncertainty at the beginning
 - Not all areas are critical to the architectural concerns
 - You'll need to learn how to distinguish *critical* uncertainty from non-critical uncertainty
 - You'll also need to learn how to reduce critical uncertainty

What Makes This Project Hard?

- The hard part is NOT any single feature
 - Although some of the features are complex
- The hard part is simultaneously achieving:
 - Correctness
 - Real-time performance
 - A feature rich display
 - Robustness to noise
 - ...
- This inevitably involves making tradeoffs ...
 - Understanding what the optimal tradeoffs are (within the given constraints) is hard
 - And requires a disciplined approach

Biggest Mistake: The Coding Trap

- A common strategy for teams is to organize responsibilities in terms of “sub systems”
 - E.g. desktop app, raspberry pi, ...
- The teams then decompose responsibilities and begin coding
- Once basic functionality has been completed the teams will begin testing QA concerns such as performance

Coding Trap II

- Why is this an issue?
 - Once the implementation has begun, the options for addressing QA concerns are reduced
- In other words you either have to rework the solution, or live with decisions that were made in the implementation
- Some of those decisions inevitably impact the QAs you are trying to address (inadvertently)

Architectural Drivers

- This project is not defined by functionality alone
- In fact, the quality attribute drivers are often more important (and more challenging to achieve)
- Additionally, projects like this come with many constraints
 - Timeline and resources are certainly a constraint
 - But you also have an existing system that can limit the options available to you

Tradeoffs Are Inevitable

- Rarely are you trying to achieve quality attributes in isolation
- You are typically trying to achieve multiple quality attributes
 - As well as multiple functions that will impact your QA concerns
- This almost always results in tradeoffs (decisions that support one concern inhibit another)
- Determining the optimal set of tradeoffs is the trick

Reducing Uncertainty

- There are many risks in this project:
 - This is a relatively complex project
 - Newly formed teams
 - Likely a new domain
 - Tight timeline
- You won't have time to redo many decisions
- This means you need to validate assumptions and reduce uncertainty early
- How can you do this?
 - Experimentation can help

Experimentation

- What is experimentation?
 - Experimentation is the process of testing ideas, hypotheses, or methods to discover their effects, validate assumptions, or uncover the unknown
- What does that mean in these kinds of projects?
 - It starts with an understanding of what's important in the project
 - You should then ask yourself “what's needed to meet expectations”?
- Let's assume latency is an important concern ...

Open Questions

- Some basic questions come to mind:
 - What is sufficient latency (and for what capability or functionality)?
 - Under what load does the system need to support the desired latency (do the latency requirements change as the load changes)?
 - What latency does the current system support (under expected load)?
 - What are the options for improving latency?
 - How well do those options work (and what are the tradeoffs)?
 - What's the effort to implement the various options?

Prioritizing Experiments

- Clearly experiments take time and effort
 - You
 - won't have time to do everything
- Identifying and prioritizing experimentation can be an art
 - You'll want to think about this like any other allocation of resources
 - Think about it in terms of costs and benefits (ROI)
 - Ask: how much effort is this experiment? What's the anticipated value of the results (relative to other experiments)?

Team Organization

- Back to team organization ...
- How should you organize your team?
 - You really want to organize your team around the tasks you have defined
 - You do NOT want your tasks derived from the team organization
- This means you need some notion of the plan in order to inform the team structure
 - The plan could be structured in phases (where the nature of the work changes as you transition from one phase to another)
- This could mean that the team structure evolves as well ...

Other Risks ...

- You're being asked to complete a significant set of capabilities
 - Again with many other risks that could result in delay
- It's not clear (even to us) how much can actually be completed in the given timeframe
- That means there's a risk you won't finish everything
 - There are known strategies for dealing with this risk, however
 - Incremental delivery is one such strategy
- This means you plan to work in an iterative way
 - Where you're delivering a working version of the system with every iteration

Expectations of Studio Project

- You should expect to work hard, but this will be a rewarding experience as well.
 - You will need to work most weekends on course and project work.
 - We expect you to try to apply the course materials to the greatest extent as possible.
 - Mentors will help you in the application of classroom materials – *use them!*
 - DO NOT hack out a solution in the last week – try to apply new design techniques and concepts!
 - Reflect constantly: what went well, what could we have done better, what will we do next time.

Deliverables I

- There will be 3 milestones for this project
- The deliverables for each milestone are as follows:
 - Milestone 1:
 - Development plan
 - Architectural drivers
 - Draft architectural approaches
 - List of identified risks
 - Experimentations planned/results from early experiments

Deliverables II

- Milestone II
 - Updated development plan
 - Updated design
 - Remaining technical risks
 - Experimentation plan/results
- Milestone III
 - System demo
 - Final presentation

Project Grading

- Each milestone of the project will be graded
- The weighting will be:
 - Milestone I: 25%
 - Milestone II: 25%
 - Milestone III: 50%
- For milestone I & II we will check for:
 - How well the team implements architectural practices
 - Are the drivers actionable
 - Do the drivers reflect the goals of the system
 - Has the team identified the design decisions that will impact the drivers
 - Has the team sufficiently investigated critical areas to expose risk
 - Does the team understand the critical aspects of their design
 - How well does the development plan support the overall goals
 - Does the implemented system comply with the intended architecture

Project Demo

- For the demo we will look at the following criteria:
 - Completion of required capability
 - Robustness/correctness of the system
 - Speed of analysis
 - Display speed
 - Measurement accuracy
 - Error detection

Resources Available

- Resources are available to support the teams
- Each team will have access to:
 - Mentors
 - Dan Plakosh, Jason Popowski, Stephan Beck (product owners)
- Dan, Jason, and Steve have developed the starter code and knows the technology, risks, and requirements
 - Any questions on the specifics of these should be addressed to Dan, Jason, and Steve
- Mentors can help with project execution
 - They will gladly review plans and strategies, help guide execution of architectural practices, and provide architectural advice and support

Suggestions

- Think carefully about upfront work
 - Experimentation should be used to uncover technical risks/validate design assumptions
- Make sure goals of the project are understood
 - How will “success” be evaluated?
 - Do the risks/experiments relate to these success criteria?
- Think carefully about work breakdown
 - How will tasks be assigned?
 - What is the “definition of done” for the tasks?
 - How are the tasks interdependent?
 - How will you ensure information is exchanged when/how it’s needed?